

**CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
CAMPUS TIMÓTEO**

Jonas Camargo

RELATÓRIO SOBRE O INTERPRETADOR

Timóteo

2025

Jonas Camargo

RELATÓRIO SOBRE O INTERPRETADOR

Relatório apresentado ao professor da disciplina Laboratório de Compiladores do Departamento de Computação e Construção Civil (DCCTM), curso de Engenharia de Computação do Centro Federal de Educação Tecnológica de Minas Gerais.

Timóteo

2025

Sumário

1	UM PROGRAMA INTERPRETADOR	3
1.1	Fases de Implementação de um Interpretador	3
2	AS BUILT	6
2.1	Estrutura e Implementação do Projeto	6
2.1.1	Modulos Produzidos	6
2.1.2	Fases de Léxico e Sintático Produzidas	7
2.2	Limitações e Erros de Projeto	8
2.2.1	Principais Limitações do Código	8
2.2.2	Principais Erros de Projeto e Soluções Propostas	9
2.3	A Contribuição das Regras Gramaticais na Implementação	9
2.4	Relatório de Bordo: Dificuldades de Desenvolvimento	10
3	OTIMIZAÇÕES E EVOLUÇÃO DO ANALISADOR SINTÁTICO	12
3.1	Otimizações Desejáveis para a Análise Sintática	12
3.2	Geradores Automáticos de Analisadores Sintáticos	12
	Referências	14

1 Um programa interpretador

“Não se possui o que não se compreende”.

Johann Goethe

Este capítulo aborda a natureza e a funcionalidade de um programa interpretador, detalhando suas fases de implementação. Um programa interpretador executa o código-fonte diretamente, linha por linha, sem a necessidade de uma compilação prévia para código de máquina. Isso contrasta com um compilador, que traduz o código-fonte completo para um executável antes da execução. Embora seja comum dizer que uma linguagem é "interpretada" (como Python), tecnicamente, toda linguagem de programação passa por um processo de compilação, sendo sua implementação mais conhecida através de um interpretador ou um compilador de fato.

A principal vantagem de um interpretador reside em sua flexibilidade e portabilidade, permitindo que o mesmo código seja executado em diferentes plataformas sem a necessidade de recompilação. No entanto, em termos de desempenho, a execução pode ser mais lenta do que a de um programa compilado, uma vez que cada linha de código precisa ser analisada e executada em tempo real. As fases essenciais para a implementação de um programa interpretador são intrinsecamente análogas às de um compilador, pois ambos os sistemas devem, fundamentalmente, compreender e processar o código-fonte. A distinção crítica reside no fato de que, ao invés de produzir um código executável final, o interpretador executa as ações e comandos à medida que as fases de análise são concluídas.

1.1 Fases de Implementação de um Interpretador

Um programa interpretador, assim como um compilador, passa por diversas fases para processar e executar o código.

A primeira fase é a Análise Léxica, também conhecida como Scanner. Sua função primordial é ler o código-source caractere por caractere e o agrupar em unidades significativas, chamadas tokens. Por exemplo, a linha de código `int idade = 20;` seria segmentada em tokens como `int` (identificado como uma palavra-chave), `idade` (reconhecido como um identificador), `=` (um operador), `20` (um literal inteiro) e `;` (um delimitador). Durante este processo, elementos como espaços em branco e comentários são reconhecidos, mas são geralmente descartados por não influenciarem diretamente na lógica do programa. O resultado dessa fase é uma fila de tokens e a atualização da tabela de símbolos, que armazena informações sobre os identificadores encontrados.

A segunda fase é a Análise Sintática, ou Parser. Esta etapa recebe a fila de tokens gerada pelo analisador léxico e tem como principal objetivo verificar se a sequência desses tokens obedece rigorosamente às regras gramaticais da linguagem em questão. Para tal, o parser tenta construir uma *Árvore Sintática Abstrata (AST)*, que serve como uma representação

hierárquica da estrutura do código-fonte. Caso a sequência de tokens não consiga ser organizada em uma estrutura gramatical válida de acordo com as especificações da linguagem, um erro de sintaxe é prontamente reportado. As regras gramaticais que guiam este processo são comumente formalizadas utilizando notações como a *Backus – Naur Form (BNF)*.

A Análise Semântica constitui a terceira fase do processo de interpretação. Seu propósito é ir além da mera verificação gramatical, focando no significado do código e na sua consistência. Nesta fase, são realizadas verificações cruciais, como a compatibilidade de tipos em operações (por exemplo, garantir que uma operação aritmética não seja realizada entre um número e um texto), a validação de que todas as variáveis utilizadas foram devidamente declaradas e a correta resolução do escopo dessas variáveis. Para auxiliar nessas verificações e para obter informações detalhadas sobre os identificadores presentes no código, a análise semântica faz uso intensivo da tabela de símbolos.

A Geração de Código Intermediário é uma fase opcional, mas frequentemente implementada, especialmente em interpretadores mais complexos. O objetivo desta etapa é criar uma representação do programa em uma linguagem mais simplificada e que seja independente da máquina específica onde o código será executado. Embora compiladores tradicionais utilizem esta fase para facilitar otimizações antes da geração do código executável final, em interpretadores, o código intermediário pode ser uma forma mais refinada e pronto para ser diretamente executado pelo motor de execução do interpretador. Essa abstração pode simplificar o processo de execução e permitir certas otimizações antes da execução final.

A fase de *Otimização* também pode ser aplicada em interpretadores, embora sua natureza e momento de ocorrência possam variar. Seu objetivo é aprimorar o código – seja ele a Árvore Sintática Abstrata, o código intermediário, ou até mesmo durante a execução em interpretadores JIT – visando a uma execução mais rápida e ao uso mais eficiente dos recursos do sistema. Em interpretadores avançados, como aqueles que empregam a compilação *Just – In – Time (JIT)*, as otimizações ocorrem dinamicamente, em tempo de execução. O compilador JIT identifica e compila trechos quentes do código, aquelas partes que são frequentemente executadas, para código nativo altamente otimizado, aprendendo e se adaptando ao comportamento real do programa durante sua execução. É bom pontuar que a otimização não é uma fase à parte e, sim, uma fase que acompanha todo o processo.

Finalmente, a fase de *Execução* é onde a principal distinção do interpretador se manifesta, contrastando com a Geração de Código Final de um compilador. Em vez de produzir um arquivo executável autônomo, o interpretador executa as instruções diretamente a partir da AST ou do código intermediário. Este processo ocorre de forma contínua, com as fases de análise e execução podendo se sobrepor ou ocorrer sequencialmente. O interpretador processa e executa o código à medida que o analisa, sem a necessidade de um passo de compilação completo antes de iniciar a execução.

Sobre ambiguidade, um interpretador de linguagem natural contorna o problema da ambiguidade ao reduzir drasticamente o escopo de interpretação. Em vez de tentar compreender a linguagem em sua totalidade, ele opera com base em uma gramática formal e um contexto extremamente limitados, focados em tarefas específicas, como buscar atributos de

um documento (tamanho, formato, título). Ao restringir as frases válidas a um pequeno conjunto de padrões pré-definidos, o analisador sintático consegue gerar uma árvore de derivação única e determinística para as entradas que reconhece, tratando qualquer outra formulação como um erro de sintaxe. Dessa forma, a ambiguidade é resolvida não pela compreensão profunda, mas pela limitação do domínio, garantindo que uma frase como "Qual o tamanho do documento?" corresponda a uma única ação, ignorando outras possíveis interpretações que estariam fora do escopo do sistema.

2 As Built

Este capítulo detalha as atividades de implementação do interpretador de linguagem natural para comandos de documentos, abordando a arquitetura das classes desenvolvidas, o fluxo de processamento léxico e sintático, as principais limitações e os desafios de projeto enfrentados. Também são propostas soluções para o aprimoramento do sistema e uma reflexão sobre a interação entre a abrangência das regras gramaticais e a eficácia da implementação.

2.1 Estrutura e Implementação do Projeto

O projeto do interpretador foi concebido com uma arquitetura modular, dividida em componentes Python, cada um encapsulando uma fase específica do processo de interpretação, em analogia às etapas de um compilador tradicional. Esta separação de responsabilidades buscou facilitar o desenvolvimento, a manutenção e a compreensão do fluxo de dados através do sistema. Foi utilizada apenas uma classe, que seria responsável pelo parser. A decisão de optar por esta classe foi a facilidade de gerir estados, diferentemente do que aconteceria caso eu implementasse de forma modular.

2.1.1 Módulos Produzidos

A implementação do interpretador é orquestrada por meio de quatro módulos Python principais: `config.py`, `linguistic_processing.py`, `syntactic_analyzer.py` e `main.py`. Cada um desses módulos define classes e funções essenciais que colaboram para o processamento da entrada do usuário.

O módulo `config.py` atua como o repositório central para as definições estáticas do interpretador. Ele contém a definição da GRAMMAR, uma estrutura de dados que representa as regras formais da linguagem que o interpretador compreende. Cada regra é um dicionário que especifica seu `rule_name`, `type` (ex: `'pergunta'`, `'atribuicao'`) e um `pattern` de tokens esperado. Os padrões são sequências de tuplas, onde cada tupla indica o tipo de token (`'KEYWORD'` para palavras literais ou um não-terminal como `'`) e seu valor. Além disso, `config.py` hospeda `NON_TERMINALS_VALIDATORS`, um dicionário que mapeia cada não-terminal a uma lista de valores permitidos ou a uma função de validação (como uma lambda para `isdigit`), fundamental para a fase de análise semântica. Este módulo também gerencia o carregamento do modelo `spaCy` para processamento linguístico avançado, com tratamento de erros para garantir que as dependências estejam disponíveis.

O módulo `linguistic_processing.py` é responsável pelas etapas de pré-processamento da linguagem natural, que seriam equivalentes à fase de análise léxica e partes da análise semântica. Ele contém as funções `get_tokens_for_parser` (anteriormente chamada `tokenize` na versão mais antiga), que transforma o texto bruto em uma sequência estruturada de tokens, e `update_symbol_table`, que constrói e mantém a tabela de símbolos do interpretador. As

estruturas de dados utilizadas incluem deque da biblioteca collections para a Fila de Tokens, garantindo operações eficientes de adição e remoção em ambas as extremidades, e uma lista padrão do Python para a Tabela de símbolos, que é mantida ordenada alfabeticamente.

A classe `SyntacticAnalyzer` (definida em `syntactic_analyzer.py`) é o coração da fase de análise sintática. Ela é instanciada com a gramática e os validadores de não-terminais, permitindo-lhe verificar a correção estrutural da entrada do usuário. A classe gerencia o estado da conversa (`self.context`) através de um dicionário, que pode ser `IDLE` (ocioso) ou `AWAITING_INPUT` (aguardando entrada). Esse estado é crucial para lidar com as interações multi-turno, onde o interpretador solicita informações adicionais para completar um comando. Internamente, o parser constrói a Árvore Sintática Abstrata (AST) como um dicionário Python, onde as chaves representam os tipos de elementos gramaticais e os valores são os tokens correspondentes extraídos da entrada. Embora simples, essa estrutura de AST serve como uma representação semântica compacta do comando reconhecido, parcialmente pronta para ser interpretada por uma fase posterior de execução.

Por fim, o módulo `main.py` serve como o ponto de entrada e o orquestrador do interpretador. Ele inicializa o modelo spaCy, o parser e a tabela de símbolos. A função `main_loop` simula o ambiente de linha de comando, recebendo a entrada do usuário e coordenando o fluxo através das fases léxica e sintática. Os resultados da análise sintática (a AST ou mensagens de erro) são impressos no console, fornecendo feedback imediato ao usuário.

2.1.2 Fases de Léxico e Sintático Produzidas

A implementação das fases de análise léxica e sintática é o coração do projeto, transformando a entrada em linguagem natural em uma estrutura compreensível pelo sistema. Na fase de Análise Léxica, o processo inicia-se conceitualmente com a função `lexical_check`, responsável pela validação de caracteres. Esta etapa assegura que a entrada do usuário contenha apenas caracteres válidos para o nosso domínio, removendo quaisquer símbolos ou caracteres especiais que não estejam explicitamente permitidos. Após essa validação inicial, a função `get_tokens_for_parser` (que substituiu a função `tokenize` original) entra em ação. Utilizando as capacidades do spaCy para processar o texto, esta função é encarregada de quebrar a sentença em unidades básicas, os tokens. Um aspecto crítico dessa implementação é a gestão de stopwords (palavras comuns como "o", "de", "para") e a preservação de palavras-chave da gramática. Foi implementada uma lógica que exclui as palavras-chave gramaticais do conjunto de stopwords, garantindo que termos como "Qual", "documento", "formato" sempre cheguem ao analisador sintático, mesmo que sejam stopwords na língua portuguesa. O resultado final desta fase é uma Fila de Tokens (implementada como um deque), que serve como entrada para a próxima fase.

Concomitantemente à tokenização, a função `update_symbol_table` contribui para a fase léxica e também para uma etapa de pré-análise semântica. Ela processa cada token validado para construir e manter uma Tabela de símbolos. O objetivo é armazenar representações únicas e canônicas das palavras semanticamente relevantes encontradas no texto. Para isso, o spaCy é empregado para realizar a lematização, que transforma as palavras em seus lemas

(forma base do dicionário, "compilando-> "compilar"). Essa abordagem é preferível ao stemming, pois a lematização preserva a significância da palavra. Além da lematização, a função utiliza a Distância Levenshtein para verificar a similaridade entre novos lemas e os já existentes na tabela de símbolos. Se um novo lemma for considerado muito similar a um já existente (abaixo de um limiar de similaridade), ele é ignorado para evitar entradas redundantes ou erros de digitação, contribuindo para a unicidade e a qualidade da tabela.

A fase de Análise sintática é executada pela classe `SyntacticAnalyzer`. O método `parse` atua como o controlador principal, que decide se a entrada é um novo comando ou a continuação de um comando anterior (resposta a uma pergunta). Para novos comandos, temos o `_parse_new_command`, que itera sobre as regras gramaticais definidas no `config.py`. O método `_match_pattern` compara a fila de tokens de entrada com o padrão de cada regra. Ele distingue entre `KEYWORD` (que exige uma correspondência exata) e não-terminais (onde qualquer token é aceito, assumindo que a validação semântica ocorrerá posteriormente). O resultado da correspondência pode ser `PERFECT_MATCH`, indicando que a frase se encaixa totalmente em uma regra; `PARTIAL_MATCH`, significando que a frase se encaixa no início de uma regra, mas falta um elemento (disparando um pedido ao usuário); ou `NO_MATCH`, indicando que nenhuma regra foi reconhecida.

Quando um `PARTIAL_MATCH` ocorre, o estado do parser muda para `AWAITING_INPUT`, e o interpretador salva o estado da AST parcial e o tipo de elemento esperado. A próxima entrada do usuário é então processada por `_complete_previous_command`, que tenta validar a resposta do usuário contra o tipo de elemento que faltava (com `NON_TERMINALS_VALIDATORS`) e, se válida, completa a AST. Essa mecânica de estado permite uma interação contextual simplória, mas eficaz para o escopo do protótipo.

2.2 Limitações e Erros de Projeto

2.2.1 Principais Limitações do Código

Apesar de sua funcionalidade básica, o interpretador, em sua forma atual, enfrenta limitações significativas que impactam sua robustez e flexibilidade na compreensão da linguagem natural. A principal restrição reside na estratégia de parsing utilizada. O método `_match_pattern` da classe `SyntacticAnalyzer` implementa uma correspondência de padrões estritamente sequencial. Isso significa que a ordem e a presença de cada token no padrão da gramática devem ser seguidas à risca. Consequentemente, a gramática definida em `config.py` é, por necessidade, extremamente restritiva e rígida. Ela aceita apenas frases com uma estrutura fixa e pré-determinada para cada comando. Qualquer desvio, por menor que seja, da sequência exata de palavras-chave e não-terminais definida nos padrões resultará em um `NO_MATCH`. Essa rigidez limita severamente a usabilidade e a "inteligência" percebida do interpretador, tornando-o incapaz de inferir comandos que não correspondam exatamente a um de seus modelos.

Adicionalmente, a validação dos não-terminais é feita de forma bastante estática. Para tipos de formato ou nomes de arquivos, são utilizadas listas hardcoded de valores permitidos.

Embora a validação de tamanho seja feita por uma função lambda que verifica se é um dígito, ainda é uma abordagem muito rudimentar. Isso significa que o interpretador não tem a capacidade de consultar fontes de dados dinâmicas (como um sistema de arquivos para verificar nomes de documentos existentes ou uma API para tipos de arquivos) para validar a semântica da entrada do usuário. Essa limitação reduz a aplicabilidade do interpretador a um conjunto muito pequeno e fixo de cenários.

2.2.2 Principais Erros de Projeto e Soluções Propostas

O principal problema encontrado foi na captação dos lemas. Inicialmente, ao utilizar a biblioteca NLTK para fazer a lematização, notei que a palavra perdia seu valor semântico. Por exemplo, quando o interpretador recebia a palavra “compilador”, ela seria salva como “compilar”. Isso levantou a questão: como eu poderia diferenciar “compilador” de “compilar” se tudo vira “compilar”? A solução foi utilizar a biblioteca spaCy. Quando é executado o `doc = nlp(user_input)`, o spaCy não apenas separa as palavras, mas também realiza uma análise gramatical completa. Para cada token, ele atribui o texto original (`token.text`), a forma base (`token.lemma_`) e, crucialmente, a Classe Gramatical (`token.pos_`), como NOUN (substantivo) ou VERB (verbo), permitindo assim a distinção entre os termos.

Apesar disso, por motivos de complexidade, ainda não explorei a fundo o spaCy. Porém, ao dar continuidade na fase de execução, isso será necessário.

Para aprimorar a validação dos não-terminais, a solução seria integrar o interpretador com fontes de dados dinâmicas. Em vez de listas hardcoded em `NON_TERMINALS_VALIDATORS`, os validadores poderiam ser funções que consultam um banco de dados simulado de documentos, um serviço web para formatos de arquivo ou até mesmo o sistema de arquivos local.

Outro ponto crítico é o tratamento de erros. Atualmente, o interpretador simplesmente informa “Não entendi” ou “Resposta inválida”. Uma solução de aperfeiçoamento seria implementar um mecanismo de sugestões ou correções de erros, similar ao que ocorre em compiladores de linguagens de programação. Ao invés de apenas falhar, o sistema poderia tentar identificar a regra gramatical mais próxima da entrada do usuário (talvez utilizando a distância Levenshtein não apenas para lemas, mas para a sequência de tokens completa) e sugerir a sintaxe correta ou os valores esperados. Apesar de que essa melhoria seria de alta complexidade e inviável para o contexto do trabalho.

2.3 A Contribuição das Regras Gramaticais na Implementação

A definição e a abrangência das regras gramaticais possuem um impacto direto e profundo na complexidade e na eficácia da implementação de um interpretador de linguagem natural. No contexto da nossa implementação, as regras gramaticais, conforme especificadas no módulo `config.py` e detalhadas em `Exemplo de regras gramaticais.md`, são predominantemente limitadas e rígidas.

Essa característica das regras tem uma contribuição clara para a implementação de um analisador sintático que é forte (no sentido de ser preciso e determinístico) para os casos exa-

tos que ele foi projetado para reconhecer. Ou seja, quando a entrada do usuário corresponde perfeitamente a um dos padrões definidos, o parser é eficaz em identificar a regra, extrair os elementos e construir a AST correspondente. Este comportamento é análogo ao de sistemas como a Alexa ou outros assistentes de voz, que são extremamente eficientes em responder a comandos bem estruturados e diretos ("Alexa, acenda a luz da sala"). Eles possuem uma gramática interna que, embora possa ser grande, é finita e focada em padrões de comando específicos. Para esses casos, a correspondência é rápida e precisa.

No entanto, essa mesma limitação das regras gramaticais prejudica drasticamente a capacidade do nosso interpretador de lidar com a variabilidade inerente à linguagem natural humana. A linguagem humana é caracterizada por sua ambiguidade, flexibilidade e redundância. Um usuário pode expressar a mesma intenção de diversas maneiras. Por exemplo, "Qual o tamanho do arquivo relatório.docx?" pode ter o mesmo significado que "Poderia me dizer o tamanho de relatório.docx?". Com a nossa gramática rígida, apenas a primeira frase, se estiver explicitamente definida como um padrão, seria reconhecida. Qualquer variação, como a reordenação de palavras ou a inclusão de termos adicionais, leva a um `NO_MATCH` e à mensagem genérica "Não entendi".

Isso cria uma dicotomia entre a abrangência da gramática e a eficácia da implementação. Nossa regra é muito limitada, mas o sintático é "forte" para aquele caso restrito. Em contraste, um sistema de NLP mais avançado, como o ChatGPT ou outros modelos de linguagem grandes (LLMs), opera com uma gramática implicitamente muito mais abrangente, aprendida a partir de bilhões de exemplos de texto. Isso permite que seu "sintático" seja mais fraco (no sentido de ser mais tolerante, não menos capaz) a variações, sinônimos e estruturas complexas. Ele consegue inferir a intenção mesmo com a ambiguidade e a criatividade da linguagem humana, tornando-o muito mais flexível. A lição aqui é que, para uma compreensão mais natural, a complexidade da gramática (seja ela explícita ou aprendida) precisa aumentar significativamente, exigindo abordagens de parsing e semântica mais sofisticadas.

2.4 Relatório de Bordo: Dificuldades de Desenvolvimento

O processo de desenvolvimento deste interpretador simplificado, como qualquer projeto de software, foi marcado por desafios e decisões importantes, que moldaram o design final do sistema. Este "relatório de bordo" documenta algumas das dificuldades enfrentadas e as soluções adotadas.

Um dos obstáculos iniciais e recorrentes foi a gestão de dependências Python. Lidar com bibliotecas de processamento de linguagem natural como NLTK e spaCy trouxe uma camada extra de complexidade. Não basta apenas instalar as bibliotecas via pip; muitas delas exigem o download de modelos de dados e recursos adicionais separadamente. Por exemplo, o NLTK precisa de punkt para tokenização e stopwords para filtragem, enquanto o spaCy exige o download de modelos de linguagem específicos (e.g., `pt_core_news_sm`) para lematização e outras análises linguísticas. Essa particularidade levou a diversos "problemas de dependência" em ambientes de desenvolvimento, impactando o cronograma de prototipagem devido ao tempo gasto em depuração de instalações. A solução parcial para isso foi a inclusão de um

arquivo requirements.txt detalhado para facilitar a instalação em um único comando, e a implementação de lógica de tratamento de erros no load_spacy_model para instruir o usuário sobre downloads pendentes.

Outro desafio significativo e uma decisão de projeto crucial foi a escolha da técnica de normalização léxica para a Tabela de símbolos. Tinha pensado inicialmente em utilizar o stemming, uma técnica de redução de palavras a seus radicais morfológicos, e de fato, havia código e comentários explorando o uso do RSLPStemmer do NLTK. No entanto, aqui há uma limitação inerente ao stemming: ele pode produzir radicais que não são palavras reais do dicionário ("correndo-> "corr", "compiladores-> "compil"). Para um interpretador que visa compreender a linguagem e, eventualmente, interagir com o usuário, a precisão e a interpretabilidade semântica são cruciais. Por isso, optamos por utilizar a lematização oferecida pelo spaCy. A lematização, embora possa ter um custo computacional ligeiramente maior (pois requer uma análise gramatical mais profunda para encontrar o lema correto), garante que a forma canônica da palavra seja um termo válido e com significado. Isso é vital para a precisão da Tabela de símbolos e para a posterior fase de análise semântica, que dependeria de uma base de lemas mais limpa e significativa.

Um problema específico de implementação que surgiu na fase léxica foi a remoção accidental de palavras-chave da gramática como stopwords. A função tokenize original (posteriormente refatorada para get_tokens_for_parser) aplicava o filtro de stopwords de forma indiscriminada. Isso levou a um bug onde comandos como "Qual documento..." eram quebrados, pois "Qual" e "documento" podiam ser filtradas como stopwords, impedindo que o analisador sintático encontrasse qualquer correspondência.

(0, 2) (0, 2)

3 Otimizações e Evolução do Analisador Sintático

Este capítulo discute as otimizações cruciais para aprimorar a fase de análise sintática do interpretador, explora futuras expansões da gramática para aumentar a utilidade do sistema para usuários reais e analisa o papel dos geradores automáticos de analisadores sintáticos no projeto.

3.1 Otimizações Desejáveis para a Análise Sintática

O analisador sintático atual, implementado em `syntactic_analyzer.py`, opera iterando por uma lista de regras e tentando combinar cada uma com a entrada do usuário. Para um número limitado de regras, essa abordagem é funcional, contudo, sua escalabilidade é limitada. Com o crescimento da gramática, o tempo de busca e comparação torna-se linear em relação ao número de regras, podendo gerar lentidão. Para mitigar esta questão, duas otimizações são altamente desejáveis.

Primeiramente, o agrupamento de regras por lookahead pode otimizar drasticamente a busca. Em vez de percorrer todas as regras em cada entrada, estas poderiam ser pré-agrupadas em um dicionário onde a chave é o primeiro token (a "palavra-chave" inicial) esperado. Por exemplo, todas as regras que se iniciam com "Qual" seriam armazenadas sob a chave "qual". Assim, o analisador consultaria apenas o primeiro token da entrada do usuário para selecionar um subconjunto relevante de regras para testar, transformando a busca de uma operação de tempo linear para tempo quase constante. Esta otimização prioriza a velocidade de processamento da entrada, potencialmente com um pequeno aumento no uso de memória para o armazenamento da estrutura de agrupamento.

Em segundo lugar, a melhoria na recuperação de erros é essencial para a usabilidade. Atualmente, o sistema lida com tokens ausentes no final de um comando, entrando no estado `AWAITING_INPUT`. No entanto, se um token incorreto for inserido no meio de uma frase (ex: "Qual formato tem relatório.docx?"), o sistema falha com "Não entendi". Uma otimização necessária seria a implementação de uma recuperação de erros mais robusta, que tentasse ignorar o token incorreto e continuar a análise do restante da frase. Isso permitiria oferecer sugestões mais úteis ao usuário, como "Você quis dizer 'tamanho' em vez de 'formato'?", melhorando a experiência e a tolerância a falhas. Esta melhoria adiciona complexidade ao código, mas o benefício na interação com o usuário justifica o esforço.

3.2 Geradores Automáticos de Analisadores Sintáticos

Ferramentas como ANTLR ou Lark (para Python) são geradores automáticos de analisadores sintáticos que seriam extremamente úteis para o projeto.

Sua utilidade reside em dois pilares principais. A primeira delas é a manutenção da gramática: Eles exigem que a gramática seja definida em um formato padrão (como EBNF), o que é substancialmente mais claro, conciso e fácil de manter do que a estrutura atual de lista de dicionários. A segunda, eficiência. Geradores automáticos criam algoritmos de análise otimizados (como LALR ou Earley) que são significativamente mais rápidos e eficientes do que uma abordagem manual de tentativa e erro, especialmente para gramáticas complexas.

Para melhorar o projeto, deveríamos primeiramente formalizar a gramática e migrar para um parser gerado, aproveitando a robustez e eficiência que essas ferramentas oferecem. Simultaneamente, seria crucial investir na adaptação da lógica interativa para que o sistema mantenha sua capacidade de diálogo, combinando a força dos geradores com a flexibilidade desejada para um interpretador de linguagem natural.

Referências

Abhishek Rajput. *Abstract Syntax Tree vs Parse Tree*. 2025. Disponível em: <<https://www.geeksforgeeks.org/compiler-design/abstract-syntax-tree-vs-parse-tree/>>. Acesso em: 28 de agosto 2025. Citado na página 11.

Spacy. *PNL*. 2025. Disponível em: <<https://spacy.io/>>. Acesso em: 28 de junho 2025. Citado na página 11.